

Proyecto – Entrega 1: Sistema de Programación de Horarios Académicos

Grupo 1:

Laura Natalia Olmos Parra

Laura Valentina Ospina Rengifo

Laura Palacios Ochoa

Felipe Mejía Alvarado

Facultad de Ingeniería, Universidad El Bosque

Ing. Samuel Agustín Galindo Pinzón

Bogotá D.C, Colombia

25 de septiembre de 2026

Tabla de Contenido

Contextualización del Problema 3
Estado del Arte 5
Análisis de Restricciones (Enfoque de Simon) 11
Diagrama de Contexto 15
Requerimientos del Sistema de Programación de Horarios Académico 15
Requerimientos Funcionales 15
Requerimientos No Funcionales 16
Definición y Justificación del Alcance (MVP) 17
Módulos Funcionales 17
Módulo de Datos y Parametrización 17
Módulo de Validación de Datos de Entrada 17
Motor de Restricciones 18
Motor de Búsqueda y Generación de Horarios 19
Módulo de evaluación de restricciones blandas 20
Interfaz Web 20
Persistencia y Consulta de Resultados 20
Alcance excluido del MVP 21
Criterios de aceptación y verificación 22
Matriz de Trazabilidad Preliminar 23
EDT / WBS y cronograma 24
Repositorio GitHub 25
Anexo de Transparencia de Uso de IA 25
Referencias 26

Contextualización del Problema

La programación de horarios académicos implica coordinar simultáneamente recursos y condiciones que dependen entre sí. Cada clase debe ubicarse en un momento y un espacio determinados, pero esa decisión está condicionada por la disponibilidad de los docentes, las características y capacidad de las aulas, el número de grupos, la cantidad de sesiones semanales de cada asignatura y las posibles coincidencias entre actividades académicas. Por esta razón, construir un horario en encontrar una configuración en la que múltiples asignaciones sean compatibles entre sí.

En los semestres del quinto al séptimo del Programa de Ingeniería de Sistemas diurno de la Universidad El Bosque, la universidad debe organizar 22 espacios académicos: 7 en quinto, 7 en sexto y 8 en séptimo, para un total de 51 créditos académicos. Sobre esta población deben considerarse posteriormente el número real de estudiantes matriculados, los grupos o secciones abiertos y los docentes disponibles para cada asignatura, información que deberá ser confirmada con el Programa.

La complejidad surge porque cada actividad académica no tiene una única alternativa de programación. Los cursos pueden requerir entre una y 4 sesiones semanales, cada una con una duración de dos horas, y deben ubicarse dentro de las franjas institucionales disponibles. Simultáneamente, un docente no puede atender dos clases en la misma franja, una misma aula no puede ser utilizada por dos grupos al mismo tiempo y determinados cursos pueden requerir espacios con características particulares, como computadores o mobiliario específico. También existen condiciones relacionadas con la capacidad de los grupos y preferencias de programación, como evitar días consecutivos para las sesiones de una misma asignatura.

Esto hace que la creación de horarios posea una complejidad combinatoria considerable. Si se representan las sesiones que deben programarse como *S*, cada una puede tener múltiples alternativas de franja horaria, aula y, dependiendo de la organización académica, docente elegible. De forma simplificada, si una sesión tuviera *T* franjas posibles y *A* aulas compatibles, existirían hasta *(T×A)^S* combinaciones potenciales antes de considerar las restricciones. El número efectivo es menor porque muchas combinaciones son inválidas, pero precisamente determinar cuáles deben descartarse obliga a comprobar múltiples condiciones de manera simultánea. Por ello, el espacio de alternativas crece rápidamente a medida que se incorporan grupos, docentes, aulas y restricciones.

El problema tampoco se limita a determinar si una asignación individual es válida. El horario debe evaluarse como un conjunto: una alternativa aparentemente adecuada para una clase puede provocar posteriormente un conflicto de docente, aula o grupo, haciendo necesario modificar decisiones anteriores y explorar otras combinaciones. A esto se suma que algunas condiciones son obligatorias, mientras que otras representan preferencias que permiten comparar diferentes horarios factibles y determinar cuáles ofrecen una mejor distribución.

Por estas características, una hoja de cálculo puede servir para **r**egistrar, consultar o visualizar información, pero resulta limitada como mecanismo para resolver el problema. La generación del horario requiere representar formalmente las restricciones, comprobar automáticamente los conflictos entre asignaciones, explorar alternativas cuando una combinación deja de ser viable y mantener reglas y parámetros que puedan modificarse sin reconstruir manualmente toda la programación. En consecuencia, se requiere una solución de ingeniería de software capaz de gestionar los datos y ejecutar de manera sistemática la lógica necesaria para generar y validar los horarios de los semestres quinto, sexto y séptimo.

Estado del Arte

Introducción y formulación del problema

El problema de programación de horarios de cursos universitarios (University Course Timetabling Problem, UCTP) representa uno de los desafíos administrativo-académicos de mayor complejidad dentro de las instituciones de educación superior (Ceschia et al., 2023). Su propósito fundamental consiste en asignar coordinadamente un conjunto de eventos académicos (cursos, asignaturas o sesiones) a un número de aulas, laboratorios, docentes y franjas temporales, garantizando el cumplimiento simultáneo de múltiples reglas institucionales (Chen et al., 2021).

A medida que se incrementa el volumen de estudiantes, la variedad de cursos, la disponibilidad docente y las restricciones de infraestructura, el espacio de búsqueda crece de manera exponencial, lo que clasifica formalmente al UCTP dentro de la categoría de problemas de optimización combinatoria de complejidad NP-hard (Abdipoor et al., 2023; Mejía Caballero & Paternina Arboleda, 2010). En consecuencia, resulta inviable aplicar métodos exhaustivos o deterministas lineales que garanticen una solución óptima global en tiempo polinomial para todas las instancias reales.

El modelado del UCTP se fundamenta en dos tipos de restricciones (Abdipoor et al., 2023; Chen et al., 2021):

- Restricciones duras (hard constraints): Determinan la factibilidad técnica del horario y son de cumplimiento obligatorio (ej. evitar solapamientos de docentes o aulas, y garantizar la capacidad física requerida). Su violación invalida la solución (Mejía Caballero & Paternina Arboleda, 2010).

- Restricciones blandas (soft constraints): Representan criterios de preferencia y calidad operacional (ej. evitar huecos en la jornada o asignar aulas estables) (Ceschia et al., 2023). Su incumplimiento no invalida la propuesta, pero penaliza la función objetivo que mide la calidad del horario.

Revisión de Paradigmas de Optimización

Programación Matemática y Métodos exactos

El enfoque clásico para abordar un UCTP consiste en la Programación Entera (IP) y la Programación Lineal Entera Mixta (MILP). En estas formulaciones, cada decisión (como asignar una asignatura específica a un aula y período determinados) se modela mediante variables binarias, mientras que las restricciones del sistema se traducen en un conjunto de ecuaciones e inecuaciones lineales. Investigaciones como la de Hernández et al. (2008) aplicaron la programación entera para resolver la asignación conjunta de horarios y aulas en facultades de ingeniería, logrando modelos analíticos de alto rigor. De igual forma, en el ámbito latinoamericano se han desarrollado modelos de optimización entera en dos fases para separar la programación de cursos de la asignación física de salas, reduciendo sustancialmente el número de variables concurrentes.

Pese a su solvencia para demostrar la optimalidad matemática mediante solucionadores comerciales, los modelos MILP experimentan una explosión combinatoria en el número de variables binarias cuando el tamaño de la institución aumenta, lo que incrementa de manera drástica el tiempo de procesamiento y la carga computacional.

Programación por Restricciones y Motores CP-SAT

La Programación por Restricciones (Constraint Satisfaction Problems, CSP) y la Programación Lógica con Restricciones (Constraint Programming, CP) permiten modelar

explícitamente el problema mediante un conjunto de variables discretas cuyas combinaciones válidas en el dominio son acotadas por restricciones institucionales (Russell & Norvig, 2022). La exploración del espacio de estados en un CSP se ejecuta tradicionalmente mediante algoritmos de backtracking, cuya eficiencia se optimiza significativamente al integrar heurísticas de ordenamiento de variables (como Minimum Remaining Values (MRV) o grado de conflicto) y de valores (Least Constraining Value), acompañadas de mecanismos de poda temprana (forward checking y propagación de arcos) (Russell & Norvig, 2022).

Un avance determinante dentro de este paradigma es el desarrollo de solvers híbridos CP-SAT, el cual combina técnicas de propagación de dominios con solucionadores de satisfacibilidad booleana (SAT). Este enfoque permite declarar restricciones complejas del dominio académico de forma expresiva y auditable, manteniendo una alta capacidad de poda y garantizando un tiempo de respuesta altamente competitivo.

Metaheurísticas y Búsqueda Local

Ante la imposibilidad de explorar de forma exhaustiva los espacios de búsqueda en instancias de gran escala, las metaheurísticas permiten obtener soluciones de alta calidad en tiempos razonables (Abdipoor et al., 2023). Por ejemplo, Mejía Caballero y Paternina Arboleda (2010) reportan la implementación exitosa de algoritmos evolutivos y genéticos para automatizar la construcción de horarios en universidades colombianas, logrando minimizar los conflictos entre asignaturas consecutivas y optimizar la carga docente. Asimismo, Gallart Suárez et al. (2010) exploraron hibridaciones de construcción heurística tipo GRASP con Búsqueda Tabú para la asignación eficiente de horarios universitarios.

Las revisiones sistemáticas recientes confirman una marcada tendencia científica hacia las metaheurísticas híbridas, tanto colaborativas como integrativas, combinando la capacidad de

exploración global de los algoritmos poblacionales con la intensidad de la búsqueda local como Simulated Annealing o Large Neighborhood Search (LNS) (Abdipoor et al., 2023). El uso de LNS resulta especialmente atractivo para entornos universitarios reales, ya que permite modificar y reparar únicamente una región acotada de la solución previa ante contingencias de última hora, evitando regenerar el horario completo.

Sistemas Multiagente y Enfoques Distribuidos

Otra aproximación la constituyen los Sistemas Multiagente (SMA), en los cuales los actores y recursos institucionales (docentes, estudiantes y administradores de infraestructura) son modelados como agentes autónomos que interactúan en un entorno distribuido (López-Cruz, 2015). Bajo este esquema, la solución al problema de horarios emerge de procesos de negociación, subastas o propagación en vecindades colaborativas, en lugar de ser calculada e impuesta de forma rígida por un algoritmo central (López-Cruz, 2015). Aunque este enfoque refleja con fidelidad la dinámica organizacional y la toma de decisiones descentralizada en las universidades, desde la perspectiva del desarrollo de software desplaza la complejidad combinatoria hacia la arquitectura del sistema, introduciendo serias dificultades en materia de coordinación de agentes, protocolos de comunicación, concurrencia y tiempos de respuesta.

Estrategias Transversales de Descomposición

La descomposición del problema no constituye un algoritmo independiente, sino una estrategia estructural aplicable a diversos paradigmas de optimización (Lach & Lübbecke, 2012). Su objetivo fundamental es fragmentar la complejidad del UCTP separando sus dimensiones principales, habitualmente dividiendo el proceso en una primera fase de asignación temporal (días y períodos) y una segunda fase de asignación espacial (salas y laboratorios) (Lach & Lübbecke, 2012). Esta separación escalar ha sido validada tanto en benchmarks internacionales

como en instituciones latinoamericanas, permitiendo estructurar problemas complejos de forma escalonada siempre que se preserven las condiciones suficientes de factibilidad espacial durante la etapa temporal.

Tabla Comparativa de Tecnologías de Solución

Enfoque	Fortalezas	Limitaciones principales	Relación con el proyecto
Programación entera / MILP	Modelo formal; manejo explícito de restricciones y función objetivo; posibilidad de demostrar optimalidad	Formulación y tamaño del modelo pueden crecer considerablemente; requiere solver especializado	Alternativa sólida, pero añade esfuerzo matemático e integración
CSP + Backtracking heurístico	Representación directa de variables y restricciones; comportamiento comprensible y auditable; permite poda temprana	Escalabilidad limitada en instancias grandes si la búsqueda crece demasiado	Adecuado como primera estrategia del MVP acotado
Constraint Programming / CP-SAT	Potente propagación y búsqueda; apropiado para scheduling; buen potencial de escalamiento	Curva de aprendizaje y dependencia de solver	Alternativa natural para evolución del sistema
Metaheurísticas / búsqueda local	Buena exploración de espacios grandes; útiles para optimizar restricciones blandas	No garantizan normalmente optimalidad; requieren diseño y calibración	Útiles para una etapa posterior de optimización
Métodos híbridos	Combinan factibilidad y optimización; alta calidad potencial	Mayor complejidad algorítmica y de integración	Interesantes para una versión más madura
Sistemas multiagente	Representan actores distribuidos y negociación	Mayor complejidad arquitectónica, comunicación y coordinación	Poco convenientes para el alcance temporal del MVP
Descomposición	Reduce la cantidad de decisiones tratadas simultáneamente	Una mala separación puede generar subsoluciones incompatibles	Estrategia complementaria aplicable al MVP y a futuros solvers

La revisión del estado del arte evidencia una brecha sustancial entre las metodologías avanzadas publicadas en la literatura académica y la realidad de los sistemas de software implementados en las instituciones educativas (Chen et al., 2021). Mientras la investigación científica prioriza el rendimiento cuantitativo sobre conjuntos de datos estandarizados (benchmarks) mediante algoritmos híbridos complejos (Ceschia et al., 2023), los entornos universitarios reales demandan plataformas flexibles, auditables y capaces de adaptarse a reglas de negocio dinámicas y cambiantes (Chen et al., 2021). La sobreingeniería algorítmica suele dificultar la mantenibilidad del código y obstaculizar la trazabilidad de los horarios generados, lo que explica por qué muchas instituciones continúan dependiendo de procesos manuales o semi-automatizados propensos a errores.

Para resolver esta problemática de manera efectiva dentro del marco de desarrollo del Producto Mínimo Viable (MVP), diseñado para un ciclo de construcción acotado de doce semanas, se adopta una arquitectura de software desacoplada. Esta separación conceptual aísla completamente el modelo de dominio y la definición de restricciones del motor algorítmico encargado del cómputo. Bajo este diseño, el MVP implementa en su primera fase un solucionador basado en CSP con Backtracking Heurístico e inferencia de poda temprana (Russell & Norvig, 2022). Esta elección se justifica en su representación directa de las variables operativas, su comportamiento determinista e interpretable por parte de los coordinadores académicos, y su independencia de solucionadores comerciales externos, garantizando una implementación limpia, robusta y completamente funcional dentro del cronograma previsto.

Finalmente, el valor diferencial de esta propuesta radica en su capacidad de evolución técnica sin reescritura de código base. Al mantener un contrato estricto de desacoplamiento entre las capas de la aplicación, el motor de backtracking heurístico del MVP podrá ser sustituido o

complementado de forma transparente en versiones futuras por un engine avanzado tipo CP-SAT o por algoritmos de búsqueda local (LNS) (Abdipoor et al., 2023; Russell & Norvig, 2022). Esta estrategia equilibra de forma óptima la viabilidad técnica inmediata requerida por el proyecto con la escalabilidad y sostenibilidad del sistema a largo plazo ante el crecimiento institucional.

Análisis de Restricciones (Enfoque de Simon)

La planificación de horarios académicos universitarios es una instancia representativa del denominado Timetable Problem, el cual se clasifica en la literatura de ingeniería como un Problema Mal Estructurado (Ill-Structured Problem – ISP) y un Problema Perverso (Wicked Problem). Está clase de problemas se caracteriza por la ausencia de una formulacion única o definitiva, la inexistencia de un criterio de parada absoluto para declarar una solución como la óptima y una inherente indeterminación en su espacio de búsqueda.

De acuerdo con el marco conceptual propuesto por Herbert Simon (1973), la frontera entre un problema bien estructurado (WSP) y uno mal estructurada (ISP) es fluida y depende de la relación entre el dominio del problema y la capacidad del solucionador. En la práctica operativa, los problemas reales no se presentan de forma intrínsecamente estructurada, se convierten en bien estructurados únicamente mediante un proceso formal de preparación, simplificación y acotación.

Dado que el solucionador opera como un procesador serial con acceso a una memoria a largo plazo (LTM) de gran escala, que almacena reglamentos, estatutos docentes, capacidades físicas de aulas y disponibilidad, resulta impracticable cargar simultáneamente toda la información en un espacio de problemas estático desde el inicio. En su lugar, el proceso de apoya en un mecanismo de reconocimiento y evocación (noticing and evoking mechanism). Este mecanismo detecta características en el estado actual del problema y evoca secuencialmente de

la memoria nuevas restricciones, submetas y alternativas, modificando de forma dinámica el espacio de trabajo activo.

Bajo esta dinámica, el problema global de horarios se aborda mediante un proceso de descomposición modular. Al concentrarse en la asignación de un recurso específico en un momento dado, por ejemplo, verificar la granja de un docente o la capacidad de un aula, el subproblema local se transforma en un enunciado bien estructurado en lo pequeño (well-structured un the small), aunque la totalidad del sistema se mantenga mal estructurada en lo grande (Ill-Structured un the large) debido a las colisiones en inconsistencias de indias que emergen al coordinar las partes.

Para acotar este espacio de estados y lograr una computación practicable, las reglas y limitaciones evocadas del entorno institucional se categoriza formalmente en dos grupos:

- **Restricciones Duras:** Condiciones físicas temporales y legales innegociables. Su incumplimiento invalida matemáticamente la solución y genera un horario físicamente imposible de ejecutar, por ejemplo, el solapamiento de un mismo docente, aula o grupo de estudiantes en un mismo bloque horario.
- **Restricciones Blandas:** Criterios de calidad, preferencia y optimización pedagógica e institucional. Si violación no anula la validez legal del horario, pero penaliza su conveniencia y usabilidad, por ejemplo, la fragmentación de la jornada docente o el agendamiento de asignaturas en días consecutivos.

A continuación, en la Tabla 1 se consolidan las restricciones duras, mientras que en la Tabla 2 se consolidan las restricciones blandas identificadas para el sistema, derivadas formalmente del caso de estudio y de los supuestos explícitos definidos por el equipo para el MVP.

Tabla 1

Restricciones duras del sistema de programación de horarios académicos

ID	Restricción	Derivación	Justificación
RD-01	Una misma aula no puede estar asignada a dos sesiones simultáneamente	Caso de estudio	Las aulas constituyen recursos limitados y no pueden ser utilizadas por dos grupos en la misma franja. El incumplimiento genera un conflicto físico de recursos
RD-02|	Un docente no puede impartir dos sesiones en la misma franja horaria	Caso de estudio y supuesto explicito	El documento establece que los docentes tienen restricciones de horario. Se asume además que un docente solo puede atender una actividad simultáneamente
RD-03	Un grupo no puede tener dos asignaturas programadas simultáneamente	Lineamiento del proyecto	Los lineamientos clasifican el choque de estudiante como una restricción dura
RD-04	Toda sesión debe ubicarse dentro de las franjas institucionales permitidas	Caso de estudio	La oferta es de lunes a viernes de 07:00 a.m. a 10:00 p.m. y sábados de 07:00 a.m. a 01:00 p.m. No se programan clases entre 12:00 a.m. y 01:00 p.m.
RD-05	Cada sesión debe tener una duración de dos horas	Caso de estudio	El documento establece que cada sesión ocupa dos horas del horario
RD-06	Una sesión solo puede asignarse a un aula cuya capacidad sea suficiente para el grupo	Caso de estudio	Las aulas tienen capacidades diferentes y los grupos no deben superar el límite definido de estudiantes
RD-07	Una sesión que requiera características específicas debe asignarse únicamente a un aula que disponga de ellas	Caso de estudio	Se menciona aulas con computadores, sillas móviles y otras características requeridas por determinados cursos
RD-08	Un docente solo puede ser asignado a cursos que pueda orientar según su experiencia y especialidad	Caso de estudio	El documento indica que se no todos los docentes pueden asumir todos los cursos
RD-09	La programación de un docente debe respetar sus restricciones horarias particulares	Caso de estudio	Algunos docentes solo pueden dictar en determinadas franjas, por ejemplo, después de las 04:00 p.m. o 06:00 p.m.
RD-10	Cada asignatura debe contar con el número de	Caso de estudio y parámetro del sistema	El caso estable una frecuencia de entre una y cuatro sesiones

	sesiones semanales requerido		semanales, con posibilidad de parametrizar el límite superior
RD-11	El tamaño de una sesión no puede superar el límite institucional establecido	Caso de estudio	El tamaño objetivo es de un máximo 40 estudiantes, con tolerancia del 10%
RD-12	Las franjas y parámetros institucionales deben ser configurables por el sistema	Caso de estudio y requisito de implementación	El caso establece que horarios, límites y otros valores deben ser parametrizables

Tabla 2

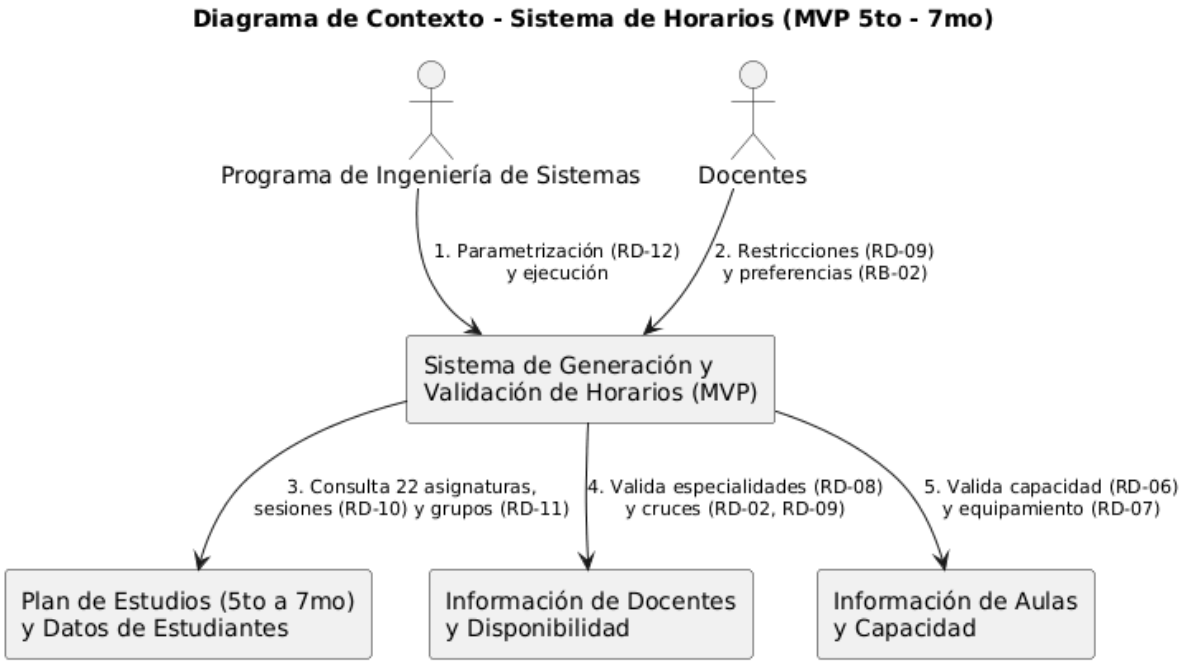
Restricciones blandas del sistema de programación de horarios académicos

ID	Restricción	Derivación	Justificación
RB-01	Evitar que las sesiones de una misma asignatura queden programadas en días consecutivos cuando tenga dos o tres sesiones semanales	Caso de estudio	El documento expresa está condición como deseable, no obligatoria
RB-02	Favorecer las franjas horarias preferidas por cada docente cuando existan varias alternativas factibles	Lineamiento del proyecto	Los lineamientos identifican las franjas preferidas por el docente como una preferencia
RB-03	Maximizar el aprovechamiento del tiempo de los docentes	Caso de estudio	La planeación de horarios contempla la maximización del aprovechamiento del tiempo de los docentes
RB-04	Evitar, cuando sea posible, distribuciones poco convenientes de las sesiones de una misma asignatura	Supuesto adicional del equipo	Se propone como criterio de calidad para seleccionar entre varios horarios factibles, sin convertirlo en condición de invalidez
RB-05	Preferir horarios que reduzcan tiempos muertos o fragmentación innecesaria en la jornada del docente	Supuesto adicional del equipo	Se incorpora como criterio de optimización, siempre que no entre en conflicto con las restricciones duras
RB-06	Preferir una distribución equilibrada de las asignaturas durante la semana	Supuesto adicional del equipo	Permite comparar soluciones factibles y seleccionar aquella con una distribución académica más conveniente

Diagrama de Contexto

Figura 1

Diagrama de contexto del sistema de programación de horarios académicos



Requerimientos del Sistema de Programación de Horarios Académico

Requerimientos Funcionales

Tabla 3

Requerimientos funcionales del sistema de programación de horarios académicos

ID	Descripción	Fuente
RF-01	El sistema debe permitir registrar y modificar las franjas horarias institucionales y los periodos de receso	RD-04, RD-12
RF-02	El sistema debe permitir configurar el tamaño objetivo, la tolerancia y el número de sesiones semanales de las secciones	RD-10, RD-11, RD-12
RF-03	El sistema debe permitir registrar las asignaturas de los semestres quinto a séptimo, incluyendo créditos y sesiones semanales requeridas	Contextualización, RD-10
RF-04	El sistema debe permitir registrar los requisitos de infraestructura, tecnología y mobiliario de cada asignatura	RD-07

RF-05	El sistema debe permitir registrar los docentes habilitados para cada asignatura	RD-08
RF-06	El sistema debe permitir registrar la disponibilidad, restricciones horarias y franjas preferidas de los docentes	RD-09, RB-02
RF-07	El sistema debe permitir registrar las aulas disponibles, incluyendo su capacidad y características	RD-06, RD-07
RF-08	El sistema debe generar automáticamente una programación de horarios asignando sesiones, franjas, aulas y docentes elegibles	RD-01 a RD-11
RF-09	El sistema debe validar que la programación generada cumpla las restricciones duras definidas	RD-01 a RD-11
RF-10	El sistema debe evaluar las programaciones factibles según el cumplimiento de las restricciones blandas definidas	RB-01 a RB-06
RF-11	El sistema debe identificar y reportar los conflictos que impidan generar una programación factible, indicando las restricciones y elementos involucrados	Contextualización
RF-12	El sistema debe permitir modificar manualmente la franja, aula o docente de una sesión mediante interacción drag-and-drop	Entregables
RF-13	El sistema debe validar las modificaciones manuales y advertir o impedir aquellas que incumplan restricciones duras	Contextualización, RD-01 a RD-11
RF-14	El sistema debe identificar y marcar las secciones con menos de 10 estudiantes para su posterior revisión administrativa	Caso de estudio
RF-15	El sistema debe permitir consultar la programación por grupo, docente y aula	Contextualización
RF-16	El sistema debe permitir exportar la programación generada en un formato definido por el proyecto	Contextualización

Requerimientos No Funcionales

Tabla 4

Requerimientos no funcionales del sistema de programación de horarios académicos

ID	Atributo de Calidad	Requerimiento
RNF-01	Rendimiento	La generación completa del horario debe tardar ≤ 5 segundos
RNF-02	Rendimiento	La validación de un cambio manual debe tardar ≤ 500 ms
RNF-03	Fiabilidad	El 100 % de los horarios generados como factibles debe cumplir las restricciones duras
RNF-04	Eficiencia	El horario generado debe obtener el mayor nivel posible de cumplimiento de las restricciones blandas

RNF-05	Usabilidad	Al menos el 90 % de los usuarios debe poder completar las tareas principales del sistema sin asistencia
RNF-06	Modificabilidad	Una nueva restricción debe poder agregarse sin modificar los componentes que no gestionan las reglas
RNF-07	Seguridad	El 100 % de las funciones de configuración de parámetros y modificación de horarios debe estar restringido a usuarios autorizados

Definición y Justificación del Alcance (MVP)

El Producto Mínimo Viable propuesto consiste en una aplicación web para la generación semiautomizada de horarios académicos de quinto, sexto y séptimo semestre del programa de Ingeniería de Sistema de la Universidad El Bosque.

Módulos Funcionales

Módulo de Datos y Parametrización

Este componente permitirá registrar, consultar y modificar la información necesaria para ejecutar la generación de horarios. Este módulo gestionará:

- Asignaturas o secciones que serán programadas (semestre al que pertenecen, número de estudiantes, requerimientos de aula).
- Información sobre los docentes (asignaturas, disponibilidad horaria).
- Información sobre las aulas (capacidad, características).
- Días y franjas institucionales disponibles.

Módulo de Validación de Datos de Entrada

Antes de iniciar la generación, el sistema deberá comprobar que los datos sean suficientes y coherentes. Las validaciones deberán comprobar:

- Cada sección cuneta con un docente compatible asociado.
- Existencia de una frecuencia semanal válida.

- La disponibilidad horaria de los docentes está expresada dentro de las franjas configuradas.
- Los datos requeridos por el algoritmo están completos.

Cuando los datos impidan iniciar razonablemente la generación, el sistema deberá informar el problema al usuario y no ejecutar el algoritmo con información inconsistente.

Motor de Restricciones

Este componente evaluará la validez de cada asignación mediante las siguientes restricciones duras:

- **RD-01:** Impedir la asignación simultánea de una misma aula a dos sesiones.
- **RD-02:** Impedir que un docente tenga dos sesiones simultáneas.
- **RD-03:** Impedir que un mismo grupo tenga dos asignaturas simultáneas.
- **RD-04:** Permitir únicamente franjas institucionales habilitadas.
- **RD-05:** Respetar la duración definida de dos horas por sesión.
- **RD-06:** Asignar únicamente aulas con capacidad suficiente.
- **RD-07:** Asignar únicamente aulas que cumplan las características requeridas por el curso.
- **RD-08:** Verificar que el docente asociado esté habilitado para orientar la asignatura.
- **RD-09:** Respetar las restricciones particulares de disponibilidad de cada docente.
- **RD-10:** Programar el número de sesiones semanales requerido por cada sección.
- **RD-11:** Validar que la sección se encuentre dentro del límite institucional establecido.

- **RD-12:** utilizar los valores institucionales configurables definidos para la ejecución.

Las restricciones duras serán obligatorias: si una asignación incumple cualquiera de ellas, no podrá formar parte de una solución válida.

Motor de Búsqueda y Generación de Horarios

La unidad básica de asignación será una sesión de clase. Para cada sesión, el algoritmo deberá buscar una combinación válida de día, franja horaria y aula.

El motor usará una estrategia de backtracking, construyendo progresivamente un horario. Cuando una asignación produzca una situación incompatible con las restricciones duras, se realizará una poda de esa alternativa y se probará otra posibilidad. Si posteriormente una decisión impide completar el horario, el algoritmo deberá retroceder y modificar asignaciones anteriores.

Por ejemplo, si una sección requiere tres sesiones semanales, el motor no considerará la asignatura como un único bloque, sino como tres sesiones que deben ser ubicadas individualmente sin perder las relaciones correspondientes a docente, grupo y frecuencia semanal.

El resultado esperado del motor será uno de los siguientes:

1. **Horario factible:** Todas las sesiones requeridas fueron asignadas respetando las restricciones duras.
2. **Sin solución encontrada bajo los datos y parámetros suministrados:** El sistema deberá informar que no pudo construir un horario completo, en lugar de presentar como válida una solución con conflictos.

El MVP no se considerará exitoso por generar cualquier horario, sino por negarse a presentar como factible uno que incumpla las restricciones duras.

Módulo de evaluación de restricciones blandas

Habiendo garantizado la factibilidad, el sistema incorporará un mecanismo de evaluación de calidad para las soluciones candidatas.

Este módulo priorizará entre las alternativas de horario en base a las restricciones blandas. Los criterios de preferencia consistirán en evitar sesiones consecutivas de una misma asignatura, respetar preferencias horarias de los docentes, reducir tiempos muertos en la jornada docente y equilibrar la carga semanal.

Interfaz Web

La aplicación contará con una interfaz web mínima que permita al usuario:

- Ingresar, consultar y modificar los datos requeridos para la generación.
- Iniciar el proceso de generación del horario.
- Conocer el estado o resultado de la ejecución.
- Visualizar el horario generado y consultar por grupo/semestre.
- Identificar claramente las asignaciones de cada curso, docente, aula, día y franja.
- Recibir mensajes cuando exista información inválida o no sea posible encontrar una solución completa.

Persistencia y Consulta de Resultados

El sistema deberá contar con un almacenamiento persistente en base de datos de configuraciones, parámetros y horarios resultante para su posterior consulta y exportación en formato CSV o PDF.

Alcance excluido del MVP

Tabla 5

Elementos fuera del alcance (MVP) del sistema de programación de horarios académicos

Elemento fuera del alcance	Justificación
Programación de todos los semestres de Ingeniería de Sistemas	El MVP se limita a 5.º, 6.º y 7.º semestre para controlar el tamaño del espacio de búsqueda manteniendo los conflictos fundamentales del problema
Programación de otros programas o facultades	Incorporaría reglas, docentes, estudiantes y recursos adicionales que aumentarían considerablemente la dimensión del problema
Creación automática del número de secciones por asignatura	Las secciones serán datos de entrada previamente definidos. Determinar cuántas deben abrirse corresponde a una etapa distinta de planeación académica
Cierre automático de secciones por baja matrícula	El caso contempla este proceso después de la matrícula; implicaría modificar la oferta y posiblemente regenerar horarios
Reasignación individual de estudiantes entre secciones	Requeriría incorporar reglas de matrícula, historial académico, cupos y Reglamento Estudiantil
Asignación automática del docente que debe orientar cada sección	El MVP recibe la relación docente–sección previamente definida y verifica su validez; no resuelve la distribución institucional de carga docente
Planeación contractual o presupuestal de docentes	Aunque se relaciona con el aprovechamiento docente, corresponde a procesos administrativos externos al motor de horarios
Integración directa con sistemas institucionales	Los datos se administrarán mediante el sistema desarrollado o archivos/datos de prueba. No se implementarán integraciones con matrícula, Talento Humano o gestión institucional de espacios
Gestión de matrícula estudiantil	El sistema trabaja con grupos y secciones ya definidos; no inscribe estudiantes ni administra su carga académica individual
Reprogramación dinámica durante el semestre	Cambios por incapacidades, cancelaciones, mantenimiento de aulas u otros eventos posteriores a la publicación del horario quedan fuera del MVP
Optimización matemática global	Se buscará una solución factible y se utilizarán preferencias heurísticas, pero no se garantizará que el resultado sea la mejor solución posible entre todas las combinaciones existentes
Edición gráfica avanzada del horario	Funciones como drag-and-drop, historial de cambios, deshacer/rehacer o resolución interactiva avanzada de conflictos no son requisito mínimo del MVP
Aplicación móvil	La solución se desarrollará como aplicación web

Gestión completa de usuarios, perfiles institucionales y autenticación corporativa	No hace parte del núcleo combinatorio que se pretende validar y supondría incorporar infraestructura institucional adicional

Criterios de aceptación y verificación

Tabla 6

Criterios de cumplimiento del MVP del sistema de programación de horarios académicos

ID	Criterio de cumplimiento del MVP	Evidencia esperada
MVP-01	El sistema permite representar las 22 asignaturas del alcance correspondiente a 5.º, 6.º y 7.º semestre	Datos almacenados y visibles en el sistema
MVP-02	El sistema permite registrar o cargar docentes, aulas, grupos, sesiones, disponibilidades y parámetros requeridos	Datos consultables desde la aplicación o base de datos
MVP-03	Cada asignatura genera exactamente el número de sesiones semanales configurado	Horario generado y prueba automatizada/manual
MVP-04	Ningún aula aparece asignada simultáneamente a dos sesiones	Prueba específica de RD-01
MVP-05	Ningún docente aparece asignado a dos clases simultáneas	Prueba específica de RD-02
MVP-06	Ningún grupo tiene dos asignaturas simultáneas	Prueba específica de RD-03
MVP-07	Todas las sesiones se encuentran dentro de las franjas institucionales configuradas y tienen la duración definida	Inspección del horario y pruebas de RD-04/RD-05
MVP-08	Ninguna sesión es asignada a un aula con capacidad insuficiente	Prueba de RD-06
MVP-09	Los requerimientos especiales de aula son respetados	Caso de prueba con aula compatible e incompatible
MVP-10	Se respetan las disponibilidades particulares de los docentes	Caso de prueba con franja permitida y prohibida
MVP-11	El sistema no presenta como válido un horario que viole una restricción dura	Pruebas de ruptura provocando conflictos
MVP-12	Ante una alternativa inválida, el algoritmo puede descartarla y explorar otra mediante backtracking	Logs, depuración o evidencia de ejecución del algoritmo
MVP-13	Cuando existe una solución factible para los datos de prueba, el sistema puede producir y mostrar al menos una propuesta completa	Ejecución integral del sistema
MVP-14	Cuando no es posible completar el horario, el sistema informa que no encontró una solución, sin relajar silenciosamente restricciones duras	Caso de prueba deliberadamente imposible

MVP-15	Las restricciones blandas pueden influir en la valoración de las soluciones sin afectar su validez	Comparación de puntuaciones o penalizaciones
MVP-16	El usuario puede iniciar la generación y visualizar el resultado desde una interfaz web	Demostración funcional
MVP-17	El horario generado identifica como mínimo asignatura, grupo, docente, aula, día y franja	Vista final del horario
MVP-18	Los principales parámetros institucionales definidos en el alcance pueden modificarse sin cambiar el código fuente del algoritmo	Prueba modificando parámetros
MVP-19	El resultado generado puede almacenarse y consultarse posteriormente	Consulta de una ejecución guardada
MVP-20	El sistema funciona con una carga de prueba igual o superior al mínimo exigido por el curso	Ejecución documentada de prueba de carga

Criterio de éxito: se establece que una ejecución será considerada exitosa únicamente cuando se obtenga una asignación completa de las sesiones incluidas en el alcance y se cumpla el 100 % de las restricciones duras implementadas.

El cumplimiento de restricciones blandas se utilizará para caracterizar la calidad de la solución, pero no determinará su factibilidad.

Un horario con una preferencia incumplida puede seguir siendo una solución válida; un horario con una sola restricción dura incumplida no podrá ser presentado por el sistema como solución válida.

Matriz de Trazabilidad Preliminar

Tabla 7

Relaciones entre restricciones y requerimientos del sistema de programación de horarios académicos

ID Restricción	ID Requerimiento
RD-01	RF-08, RF-09, RF-13 y RNF-03
RD-02	RF-08, RF-09, RF-13 y RNF-03
RD-03	RF-08, RF-09, RF-13 y RNF-03

RD-04	RF-01, RF-08, RF-09, RF-13 y RNF-03
RD-05	RF-08, RF-09, RF-13, RNF-03
RD-06	RF-07, RF-08, RF-09, RF-13 y RNF-03
RD-07	RF-04, RF-07, RF-08, RF-09, RF-13 y RNF-03
RD-08	RF-05, RF-08, RF-09, RF-13 y RNF-03
RD-09	RF-06, RF-08, RF-09, RF-13 y RNF-03
RD-10	RF-02, RF-03, RF-08, RF-09, RF-13 y RNF-03
RD-11	RF-02, RF-08, RF-09, RF-13 y RNF-03
RD-12	RF-01, RF-02 y RNF-03
RB-01	RF-10 y RNF-04
RB-02	RF-06, RF-10 y RNF-04
RB-03	RF-10 y RNF-04
RB-04	RF-10 y RNF-04
RB-05	RF-10 y RNF-04
RB-06	RF-10 y RNF-04

EDT / WBS y cronograma

Figura 2

EDT / WBS del sistema de programación de horarios académicos

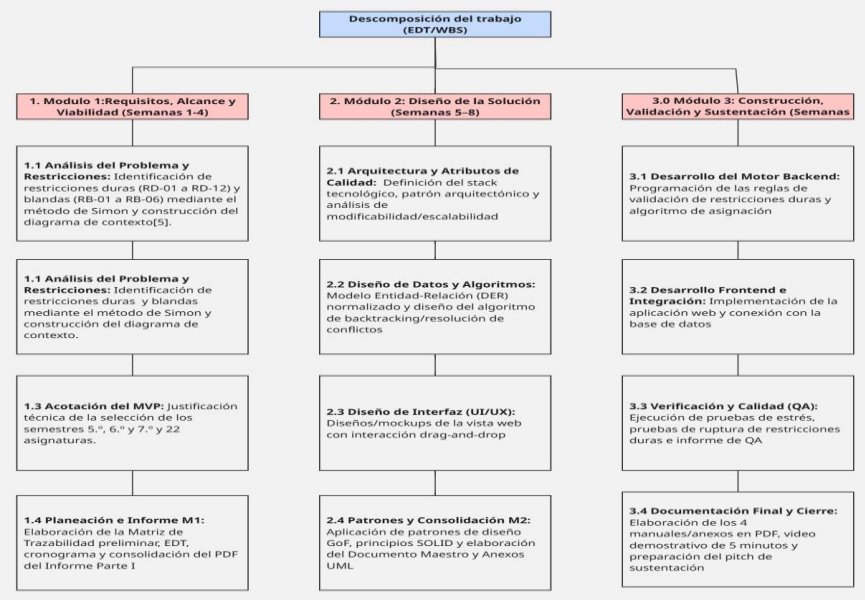
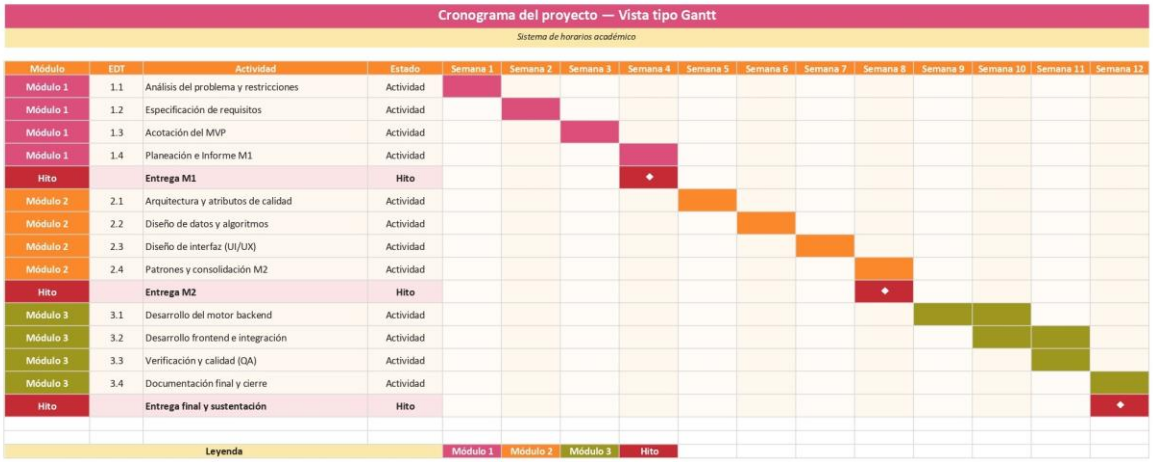


Figura 3

Cronograma del sistema de programación de horarios académicos



Cronograma del proyecto — Vista tipo Gantt

Sistema de horarios académico

Módulo	EDT	Actividad	Estado	Semana 1	Semana 2	Semana 3	Semana 4	Semana 5	Semana 6	Semana 7	Semana 8	Semana 9	Semana 10	Semana 11	Semana 12
Módulo 1	1.1	Análisis del problema y restricciones	Actividad												
Módulo 1	1.2	Especificación de requisitos	Actividad												
Módulo 1	1.3	Acotación del MVP	Actividad												
Módulo 1	1.4	Planeación e Informe M1	Actividad												
Hito		**Entrega M1**	Hito												
Módulo 2	2.1	Arquitectura y atributos de calidad	Actividad												
Módulo 2	2.2	Diseño de datos y algoritmos	Actividad												
Módulo 2	2.3	Diseño de interfaz (UI/UX)	Actividad												
Módulo 2	2.4	Patrones y consolidación M2	Actividad												
Hito		**Entrega M2**	Hito												
Módulo 3	3.1	Desarrollo del motor backend	Actividad												
Módulo 3	3.2	Desarrollo frontend e integración	Actividad												
Módulo 3	3.3	Verificación y calidad (QA)	Actividad												
Módulo 3	3.4	Documentación final y cierre	Actividad												
Hito		**Entrega final y sustentación**	Hito												

Leyenda Módulo 1 Módulo 2 Módulo 3 Hito

Repositorio GitHub

Link: https://github.com/LauraPalaciosO/Sistema-de-Programacion-de-Horarios-Academicos.git

Anexo de Transparencia de Uso de IA

Se utilizó NotebookLM para apoyar el análisis del caso de estudio y asegurarnos que los planteado represente correctamente la problemática. Además, se usó Gemini para corregir ortografía y redacción.

Referencias

Abdipoor, S., Yaakob, R., Goh, S. L., & Abdullah, S. (2023). *Meta-heuristic approaches for the University Course Timetabling Problem.* Intelligent Systems with Applications, 19, Artículo 200253. https://doi.org/10.1016/j.iswa.2023.200253

Ceschia, S., Di Gaspero, L., & Schaerf, A. (2023). *Educational timetabling: Problems, benchmarks, and state-of-the-art results.* European Journal of Operational Research, 308(1), 1–18. https://doi.org/10.1016/j.ejor.2022.07.011

Chen, M. C., Sze, S. N., Goh, S. L., Sabar, N. R., & Kendall, G. (2021). *A survey of University Course Timetabling Problem: Perspectives, trends and opportunities.* IEEE Access, 9, 106515–106529. https://doi.org/10.1109/ACCESS.2021.3100613

Gallart Suárez, J., Alva Manchego, F., Alama, N. A., & Bejarano Nicho, G. (2010). *Generación inteligente de horarios empleando heurísticas GRASP con búsqueda tabú para la Pontificia Universidad Católica del Perú.* Revista de Ingeniería Informática PUCP, 1(1), 15–23.

Hernández, R., Miranda, J., & Rey, P. A. (2008). *Programación de horarios de clases y asignación de salas para la Facultad de Ingeniería de la Universidad Diego Portales mediante un enfoque de programación entera.* Revista Ingeniería de Sistemas, 22, 121–141.

Kendall, G., & Hussin, N. M. (2004). *A tabu search hyper-heuristic approach to the examination timetabling problem at the MARA university of technology.* En E. Burke & M. Trick (Eds.), Practice and Theory of Automated Timetabling V (pp. 270–293). Springer. https://doi.org/10.1007/11593577_16

Lach, G., & Lübbecke, M. E. (2012). *Curriculum based course timetabling: New solutions to Udine benchmark instances.* Annals of Operations Research, 194(1), 255–272. https://doi.org/10.1007/s10479-010-0707-0

López-Cruz, O. (2015). *Una solución basada en agentes al problema de generación de horarios.* Revista Ingeniería, Matemáticas y Ciencias de la Información, 2(9), 73–85.

Mejía Caballero, J. M., & Paternina Arboleda, C. (2010). *Asignación de horarios de clases universitarias mediante algoritmos evolutivos.* Revista Educación en Ingeniería, 5(9), 140–149.

Russell, S., & Norvig, P. (2022). *Artificial intelligence: A modern approach* (4.ª ed. global). Pearson Education.